

Fixing Noisy, Aliased Source Art

A reusable downsample-then-upsample technique for cleaning up jagged or grainy raster images — written up from a fix applied to the *Miku Fever Match-3* character portraits, generalized for reuse on other projects.

1. The problem

A source PNG can look fine at a glance and still be full of low-level noise: every pixel carries a small amount of unwanted variation, and edges that should be clean diagonals instead stair-step or look faintly speckled. This is easy to miss by eye at 100% zoom, but it becomes obvious the moment the image is scaled up — CSS `background-size`, a canvas `drawImage()` zoom, a game engine's sprite scale factor, anything that magnifies the pixels also magnifies the noise pattern sitting inside them.

The telltale sign it's baked into the pixels (not a display/scaling artifact) is that fully *transparent*-looking regions of a PNG still carry non-zero, non-uniform alpha values when you sample them directly — e.g. a corner that should read `alpha = 0` instead reads `alpha = 3`, and no two nearby pixels agree. That's a strong signal the file itself needs cleaning up, not just a CSS tweak.

2. Diagnosing it first

Before reaching for a fix, confirm the noise is actually in the file. Sample a handful of pixels directly — corners that should be fully transparent, and a horizontal strip through an area that should be a flat color — and look at the raw values instead of just eyeballing the image:

PowerShell / .NET (System.Drawing), reads a pixel and its alpha channel directly:

```
$bmp = [System.Drawing.Bitmap]::new("portrait.png")
$corner = $bmp.GetPixel(5, 5)
"A=$($corner.A) R=$($corner.R) G=$($corner.G) B=$($corner.B)"
# A corner that should be fully transparent reading A=3
# instead of A=0, repeated across many sampled points, means
# the file has noise baked in – not just a display issue.
```

If you can, also zoom into a fine-detail region (hair strands, a thin diagonal edge) at 3–4× with nearest-neighbor scaling (so you're seeing the real pixels, not a smoothed re-interpretation of them) and look for stair-stepping or speckling directly.

3. The fix: downsample, then upsample

The reference technique for cleaning up jagged edges is supersampling: render at a much higher resolution than needed, then downsample to the target size, averaging many source pixels into each output pixel. That averaging is what smooths jaggies and noise alike.

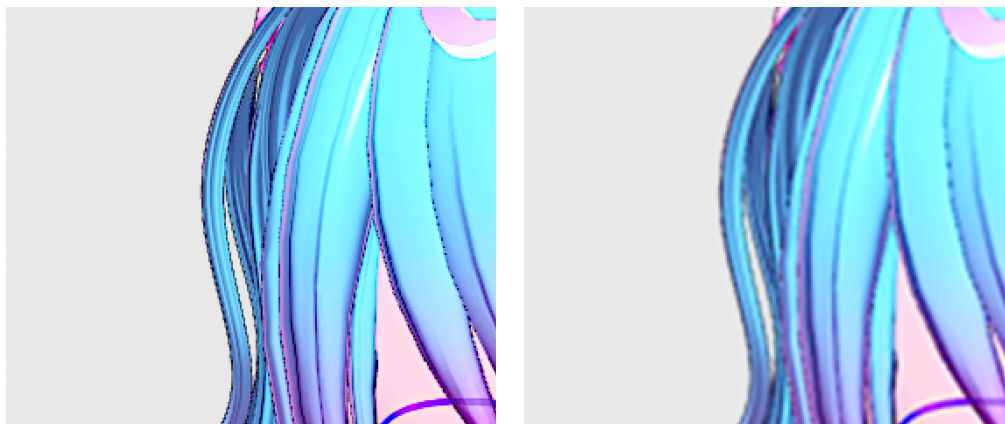
You usually can't re-render an existing raster image at higher resolution — the detail past its native resolution simply doesn't exist. But you can run the same averaging idea in reverse on what you've already got: **downsample the existing image to a smaller size first (this averages neighboring noisy pixels together), then upsample it back up with high-quality bicubic interpolation.** The intermediate downsample step is what actually removes the noise; the upsample just restores the working resolution you need.

This works because per-pixel noise is high-frequency (it varies pixel-to-pixel), while the real image content — edges, shading, silhouettes — is comparatively low-frequency (it varies smoothly over many pixels). Averaging a block of pixels together cancels out high-frequency noise far more than it blurs low-frequency content, as long as you don't downsample so aggressively that you start averaging away real detail too.

4. Before / after

Both examples below are the same crop, at the same zoom, before and after a 50% downsample-then-upsample pass ($1920 \times 1440 \rightarrow 960 \times 720 \rightarrow 1920 \times 1440$, bicubic both directions).

Fine hair-strand detail:



Before — visible stair-stepping on every strand

After — clean, smooth edges, detail preserved

Face detail (edges + fine linework together):



Before — jagged hairline, speckled edges



After — smooth, no visible detail loss

5. The recipe (PowerShell / .NET)

Self-contained — no external dependencies beyond .NET's built-in System.Drawing (GDI+), which ships with Windows. Preserves alpha transparency (Format32bppArgb) so it's safe for character art on transparent backgrounds, not just flat photos.

Load source bytes into a MemoryStream rather than [Image]::FromFile() directly if the output path can ever be the same as the input — FromFile() keeps a lock on the source file, and saving back to that same path throws "A generic error occurred in GDI+".

```
Add-Type -AssemblyName System.Drawing

function Apply-Smoothing($srcPath, $outPath, $downW, $downH, $finalW, $finalH) {
    $bytes = [System.IO.File]::ReadAllBytes($srcPath)
    $ms = New-Object System.IO.MemoryStream($bytes)
    $src = [System.Drawing.Image]::FromStream($ms)

    # Pass 1: downsample — averages neighboring noisy pixels together
    $down = New-Object System.Drawing.Bitmap($downW, $downH, `
        [System.Drawing.Imaging.PixelFormat]::Format32bppArgb)
    $g1 = [System.Drawing.Graphics]::FromImage($down)
    $g1.CompositingMode = [System.Drawing.Drawing2D.CompositingMode]::SourceCopy
    $g1.InterpolationMode = [System.Drawing.Drawing2D.InterpolationMode]::HighQualityBicubic
    $g1.PixelOffsetMode = [System.Drawing.Drawing2D.PixelOffsetMode]::HighQuality
    $g1.SmoothingMode = [System.Drawing.Drawing2D.SmoothingMode]::HighQuality
    $g1.DrawImage($src, [System.Drawing.Rectangle]::new(0,0,$downW,$downH))
    $g1.Dispose()

    # Pass 2: upsample back to the resolution you actually need
    $final = New-Object System.Drawing.Bitmap($finalW, $finalH, `
        [System.Drawing.Imaging.PixelFormat]::Format32bppArgb)
    $g2 = [System.Drawing.Graphics]::FromImage($final)
    $g2.CompositingMode = [System.Drawing.Drawing2D.CompositingMode]::SourceCopy
    $g2.InterpolationMode = [System.Drawing.Drawing2D.InterpolationMode]::HighQualityBicubic
    $g2.PixelOffsetMode = [System.Drawing.Drawing2D.PixelOffsetMode]::HighQuality
    $g2.SmoothingMode = [System.Drawing.Drawing2D.SmoothingMode]::HighQuality
    $g2.DrawImage($down, [System.Drawing.Rectangle]::new(0,0,$finalW,$finalH))
    $g2.Dispose()

    $src.Dispose(); $ms.Dispose()
    $final.Save($outPath, [System.Drawing.Imaging.ImageFormat]::Png)
    $down.Dispose(); $final.Dispose()
}

# Example: 1920x1440 source, denoise via a 50% round-trip,
# end back at the original resolution
Apply-Smoothing "portrait.png" "portrait.png" 960 720 1920 1440
```

6. Choosing how far to downsample

Downsample ratio	Result	Verdict
50% (e.g. 1920×1440 → 960×720)	Noise/jaggedness removed; fine detail (individual hair strands, eye highlights) stays crisp	Sweet spot for character art
33% (e.g. 1920×1440 → 640×480)	Even cleaner, but thin strands start blurring into each other	Only if 50% still isn't enough

Start at 50%. If the source is noisier than usual, or the final display size is small enough that losing a little fine detail won't be noticeable, try 33–40%. There's no universal correct number — always generate one test image and do a zoomed before/after comparison rather than assuming a ratio that worked on one image will be right for another.

7. Adapting this to other tools

The technique itself — downsample with high-quality interpolation, then upsample back — isn't specific to PowerShell or .NET. It carries over directly to any tool with a good bicubic/Lanczos resize:

- **ImageMagick:** `magick portrait.png -resize 50% -filter Lanczos -resize 200% out.png`
- **Python / Pillow:** `img.resize((w//2, h//2), Image.LANCZOS).resize((w, h), Image.LANCZOS)`
- **GIMP:** Image → Scale Image down (Cubic), then Scale Image back up (Cubic) — same two-pass idea, done by hand
- **Photoshop:** Image Size down with Bicubic Smoother, then Image Size back up with Bicubic Sharper

Whatever the tool, the two things that matter are: (1) use a genuinely high-quality resampling filter in both directions — nearest-neighbor or a low-quality bilinear won't do the averaging that actually removes the noise, and will often make jaggedness worse, not better — and (2) preserve the alpha channel throughout if the source has transparency, or you'll get dark or light fringing at the edges of transparent regions.